ip_idmap

Configurable ID Translation Table

Hardware IP Specification

Document ip_idmap Hardware Specification

Contents

1 Introduction **1**

1.1 Overview . . . 1

1.2 Features . . . 1

1.3 Terminology . . . 2

1.4 Conventions . . . 2

2 Configuration Parameters **3**

3 Mapping Modes **3**

3.1 Summary . . . 3

3.2 U2U: unique to unique . . . 4

3.3 S2U: same to unique . . . 4

3.4 S2S: same to same . . . 4

4 Functional Description **4**

4.1 Allocate . . . 4

4.2 Response (pop) . . . 5

4.3 Lookup (poll) . . . 5

5 Interface **6**

5.1 Clock and reset . . . 6

5.2 Allocate interface . . . 7

5.3 Response interface . . . 7

5.4 Lookup interface . . . 7

6 Timing **8**

6.1 Operation timing . . . 8

6.2 Same-cycle interactions . . . 8

6.3 Timing diagrams . . . 8

7 Micro-architecture (informative) **9**

7.1 Storage . . . 9

7.2 Free-ID pool . . . 10

7.3 Lookup structures . . . 10

7.4 S2S reference counting 10

8 Illegal Conditions **11**

9 Reset **11**

A Worked Examples (informative) **11**

A.1 Conventions 11

A.2 Side by side: the same requests in each mode 12

A.3 U2U trace 12

A.4 S2U trace 13

A.5 S2S trace 13

A.6 Interface-level view of one transaction 14

List of Figures

1 ip_idmap block diagram and interface groups. 1
2 Allocation stalled by full, released by a pop. 9
3 Response by downstream ID with a same-cycle poll. 9

List of Tables

1 Terminology. 2
2 Configuration parameters. 3
3 Comparison of the mapping modes. 3
4 alloc_drdy generation. 4
5 State update on an accepted allocation. 5
6 Effect of a response. 5
7 Poll ports. 6
8 Clock and reset signals. 6
9 Allocate signals. 7
10 Response (pop) signals. 7
11 Lookup (poll) signals. 7
12 Operation timing. 8
13 Same-cycle interaction rules. 8
14 Free-entry counter update rules. 10
15 Illegal conditions. 11
16 The same requests in each mode. 12
17 U2U trace. 12
18 S2U trace. 13
19 S2S trace. 14
20 S2S at full and at maximum reference count. 14

1 Introduction

1.1 Overview

ip_idmap is a configurable, low-latency ID translation table with a built-in free-ID allocator (free pool). It holds a fixed number of entries. Each entry binds one **downstream ID** (the entry's index) to one **upstream ID** (a value stored in the entry).

The block is used to:

- **compress** a wide or unconstrained *upstream* ID space into a small, dense *downstream* ID space when a request is issued; and
- **translate back** from the downstream ID to the original upstream ID when the response returns.

The block provides three operations — **allocate**, **response** (pop) and **lookup** (poll) — and supports three mutually exclusive **mapping modes** — **U2U**, **S2U** and **S2S**. Figure 1 shows the block and its three interfaces.

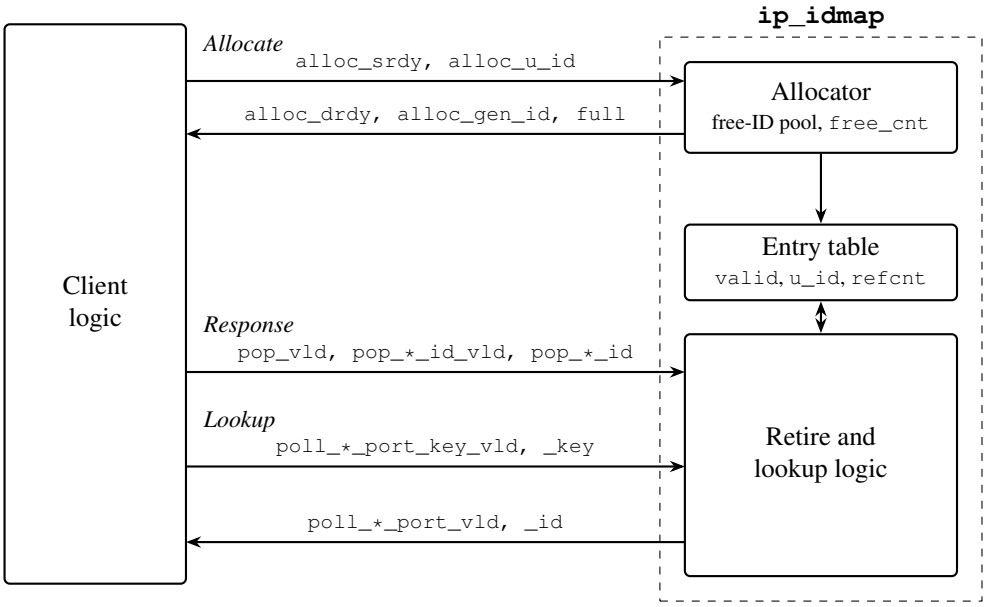


Figure 1. ip_idmap block diagram and interface groups.

1.2 Features

- Fixed depth, parameterized: NUM_ENTRIES entries, not required to be a power of two.
- Built-in free-ID pool; the requester never chooses the downstream ID.
- Zero-latency allocate: alloc_gen_id is produced in the same cycle as the handshake.
- Same-cycle lookup in both directions (u→d and d→u) on dedicated poll ports.
- Three mapping modes (U2U, S2U, S2S), selected at elaboration time and mutually exclusive.
- Reference counting in S2S mode.
- Single allocate port and single response port; throughput of one allocate and one response per cycle.
- An internal free-entry counter drives full; the count itself is not exposed as an output.

1.3 Terminology

Table 1. Terminology.

Term	Meaning
Upstream ID (u_id)	ID supplied by the requester. Wide, arbitrary, possibly sparse; `U_ID_W` bits.
Downstream ID (d_id)	ID generated by `ip_idmap`. Equal to the entry index. Dense; `D_ID_W` $= \lceil \log_2(\texttt{NUM_ENTRIES}) \rceil$ bits.
Entry	One table slot: `{valid, u_id, [refcnt]}`.
Free pool	The set of entries with `valid == 0`; the source of new d_ids.
Free count	Internal counter of free entries (`free_cnt`). Drives `full`; not an output.
Live	An entry that is allocated and not yet fully retired.
Retire	Release of an entry, or of one reference to it in S2S. Synonym of *response* and *pop*.
srdy / drdy	Source-ready / destination-ready handshake. A transfer occurs in each cycle in which both are high.

1.4 Conventions

- The words **shall**, **should** and **may** are used as follows: **shall** denotes a mandatory requirement; **should** denotes a recommendation; **may** denotes a permitted option.
- Sections marked *informative* (Section 7 and Appendix A) do not impose requirements.
- Signal directions are given from the point of view of `ip_idmap`.
- In signal names, the suffix `_vld` denotes a valid qualifier, `srdy`/`drdy` denote the source/destination-ready handshake, and `_n` denotes an active-low signal.
- In poll-port names, `x` stands for `u` or `d`.
- Widths are written `U` for `U_ID_W` and `D` for `D_ID_W`.
- Numbers are decimal unless prefixed with `0x`. Cycle numbers are counted in clock cycles of `clk`.

2 Configuration Parameters

Table 2. Configuration parameters.

Parameter	Default	Description
NUM_ENTRIES	16	Number of table entries; equal to the size of the downstream ID space.
U_ID_W	8	Upstream ID width in bits.
D_ID_W	$clog2(NUM_ENTRIES)	Downstream ID width. Derived; shall not be overridden.
FREE_CNT_W	$clog2(NUM_ENTRIES+1)	Width of the internal free-entry counter. Derived; shall not be overridden.
MODE	U2U	Mapping mode: U2U, S2U or S2S. Static, elaboration-time.
REFCNT_W	4	Reference-counter width per entry. Used only when MODE == S2S. The maximum reference count is $2^{\text{REFCNT_W}} - 1$.

3 Mapping Modes

The mapping mode is fixed at elaboration by the MODE parameter. Exactly one mode shall be active in any instance.

3.1 Summary

Table 3. Comparison of the mapping modes.

Property	U2U	S2U	S2S
Full name	Unique → Unique	Same → Unique	Same → Same
Meaning	Every live u_id is unique; 1 entry ↔ 1 u_id.	Same u_id may be live several times; **each allocation gets its own d_id**.	Same u_id is **merged** into one entry shared by all its allocations.
Entries per live u_id	0 or 1	0 to NUM_ENTRIES	0 or 1
Alloc, u_id not live	New entry from pool.	New entry from pool.	New entry from pool, refcnt = 1.
Alloc, u_id already live	**Illegal** (Section 8).	Allowed; new entry from pool.	Merge: refcnt += 1, existing d_id returned, **no entry consumed**.
Retire key	u_id **or** d_id	**d_id only**	u_id **or** d_id
Retire effect	Entry freed.	Entry freed.	refcnt -= 1; entry freed when it reaches 0.
Lookup u→d	Unique result.	**Not supported** (Section 3.3).	Unique result.
Lookup d→u	Unique result.	Unique result.	Unique result.
d_ids per u_id	1	Many	1
u_ids per d_id	1	1	1, with refcnt allocations outstanding.

3.2 U2U: unique to unique

U2U provides a strict 1:1 binding. The requester shall not allocate an upstream ID again while it is live. Because both directions are unambiguous, a response may identify its entry by either key.

3.3 S2U: same to unique

In S2U, repeated allocations of one upstream ID each receive a different downstream ID, so the d→u mapping is many-to-one while every d_id remains unique. Because a u_id can correspond to several live entries, u→d is ambiguous; therefore the downstream ID is the only usable key:

- Retire **shall** be by downstream ID only (`pop_d_id_vld`).
- Lookup **shall** be by downstream ID only (`poll_d_port`). The upstream-keyed `poll_u_port` is not supported in this mode (Section 4.3).

3.4 S2S: same to same

In S2S, allocations sharing an upstream ID share one entry. Each allocation increments the entry's reference count and each response decrements it. The entry, and its d_id, return to the free pool only when the count reaches zero. A merge does not consume a free entry and can therefore be accepted even when the pool is empty (Section 4.1).

4 Functional Description

4.1 Allocate

The allocate operation binds a new upstream ID to a downstream ID drawn from the free pool.

Handshake. The requester asserts `alloc_srdy` together with `alloc_u_id`. `ip_idmap` asserts `alloc_drdy` when it can accept the allocation. A transfer occurs in each cycle in which `alloc_srdy && alloc_drdy`. Once the requester has asserted `alloc_srdy`, it **shall** keep `alloc_srdy` asserted and `alloc_u_id` stable until `alloc_drdy` is asserted.

Result. In a transfer cycle, `alloc_gen_id` **shall** carry the assigned downstream ID. `alloc_gen_id` is valid whenever `alloc_srdy && alloc_drdy` and is a don't-care otherwise.

Ready rules. `alloc_drdy` **shall** be generated as given in Table 4. In S2S, `alloc_drdy` therefore depends combinationally on `alloc_u_id`. In S2S, when the `refcnt` of the matching entry is at its maximum, allocation of that u_id stalls until a response decrements it.

Table 4. `alloc_drdy` generation.

Mode	`alloc_drdy`
U2U, S2U	`!full`
S2S	`!full` **or** (`alloc_u_id` matches a live entry **and** that entry's `refcnt` is not at its maximum)

State update. An accepted allocation **shall** update the table as given in Table 5. The update is visible from

the next cycle.

Table 5. State update on an accepted allocation.

Mode	Case	Update
U2U, S2U	any	Selected entry: `valid` ← 1, `u_id` ← `alloc_u_id`.
S2S	new u_id	Selected entry: `valid` ← 1, `u_id` ← `alloc_u_id`, `refcnt` ← 1.
S2S	u_id live (merge)	Matching entry: `refcnt` ← `refcnt` + 1; `alloc_gen_id` is that entry's index.

Free-ID selection. The next d_id **shall** be the lowest-index free entry (find-first-zero over the inverted valid vector). It is presented on `alloc_gen_id` continuously while `!full`, so that the allocate result is available in the same cycle without a registered stage.

4.2 Response (pop)

The response operation retires a previously allocated entry, identified by its upstream ID **or** its downstream ID.

- `pop_vld` marks a response event. Responses are never back-pressured; there is no ready signal.
- Exactly one of `pop_u_id_vld` and `pop_d_id_vld` **shall** be asserted with `pop_vld`; it selects which key field is meaningful (`pop_u_id` or `pop_d_id`).
- The addressed entry **shall** be live (Section 8).

Table 6. Effect of a response.

Mode	Effect
U2U	`valid` ← 0; the d_id returns to the free pool.
S2U	`valid` ← 0; the d_id returns to the free pool. The key shall be a d_id.
S2S	`refcnt` ← `refcnt` − 1. If the result is 0, `valid` ← 0 and the d_id returns to the free pool.

Reuse timing. A freed d_id becomes available for allocation in the *next* cycle. There is no same-cycle bypass from pop to allocate.

Response translation. The counterpart ID needed to translate the response (the u_id for a d_id key, or vice versa) is obtained through the poll ports in the same cycle (Section 4.3). The pop interface itself returns no data.

4.3 Lookup (poll)

The lookup operation reads the current table contents without modifying them. Two independent, combinational poll ports are provided (Table 7).

Table 7. Poll ports.

Port	Key input	Result outputs
poll_u_port	upstream ID (poll_u_port_key)	hit flag (poll_u_port_vld) and downstream ID (poll_u_port_id)
poll_d_port	downstream ID (poll_d_port_key)	hit flag (poll_d_port_vld) and upstream ID (poll_d_port_id)

- A query is qualified by poll_x_port_key_vld. When it is 0, poll_x_port_vld **shall** be 0.
- poll_x_port_vld **shall** be 1 only when the key matches a *live* entry.
- poll_x_port_id is a don't-care when poll_x_port_vld is 0.
- Polls observe the table state at the start of the cycle. They **shall not** see an allocation or pop occurring in the same cycle.
- Poll ports are always available and never stall, nor do they interact with alloc_drdy.
- **S2U:** poll_u_port is not supported. Its outputs (poll_u_port_vld, poll_u_port_id) **shall** be tied to 0, and asserting poll_u_port_key_vld is illegal (Section 8, I8). poll_d_port behaves as described above.

5 Interface

Signal directions are from the point of view of ip_idmap. In the following tables x is u or d, U is U_ID_W and D is D_ID_W.

5.1 Clock and reset

Table 8. Clock and reset signals.

Signal	Dir	Width	Description
clk	in	1	Clock. All logic is synchronous to its rising edge.
rst_n	in	1	Active-low reset. See Section 9.

5.2 Allocate interface

Table 9. Allocate signals.

Signal	Dir	Width	Description
alloc_srdy	in	1	Requester has an upstream ID to allocate. Once asserted it shall stay asserted, with alloc_u_id stable, until alloc_drdy is seen.
alloc_drdy	out	1	ip_idmap can accept the allocation this cycle (Section 4.1).
alloc_u_id	in	U	Upstream ID to bind.
alloc_gen_id	out	D	Assigned downstream ID. Valid when alloc_srdy && alloc_drdy.
full	out	1	Free pool is empty (all entries live). Derived from the internal free-entry counter (Section 7.1).

5.3 Response interface

Table 10. Response (pop) signals.

Signal	Dir	Width	Description
pop_vld	in	1	Response event: retire one entry (one reference in S2S).
pop_u_id_vld	in	1	Retire key is pop_u_id. Illegal in S2U.
pop_d_id_vld	in	1	Retire key is pop_d_id.
pop_u_id	in	U	Upstream ID key.
pop_d_id	in	D	Downstream ID key.

5.4 Lookup interface

Table 11. Lookup (poll) signals.

Signal	Dir	Width	Description
poll_u_port_key_vld	in	1	Upstream-keyed query valid.
poll_u_port_key	in	U	Upstream ID to look up.
poll_u_port_vld	out	1	Hit: the key matches a live entry.
poll_u_port_id	out	D	Downstream ID of the matching entry.
poll_d_port_key_vld	in	1	Downstream-keyed query valid.
poll_d_port_key	in	D	Downstream ID to look up.
poll_d_port_vld	out	1	Hit: the entry at the key index is live.
poll_d_port_id	out	U	Upstream ID stored in that entry.

When MODE == S2U, the poll_u_port_* outputs **shall** be tied to 0 and poll_u_port_key_vld **shall not** be asserted.

6 Timing

All times are in clock cycles. T is the cycle in which the event occurs.

6.1 Operation timing

Table 12. Operation timing.

Event	Timing
Allocate handshake at T	`alloc_gen_id` valid in T; the entry is live and visible to polls from $T+1$.
Pop at T	The entry (or reference) is released at $T+1$; the freed d_id is allocatable from $T+1$.
Poll at T	Result valid in T (combinational from registered state).
`full`	Registered-state function; reflects the table at the start of the cycle.

6.2 Same-cycle interactions

Table 13. Same-cycle interaction rules.

Combination in cycle T	Behavior
Allocate + pop	Both proceed. If the table is full at the start of T, the allocate stalls (unless it is an S2S merge).
Allocate + poll	The poll sees the pre-allocate state.
Pop + poll	The poll sees the pre-pop state; the entry is still live.
S2S merge + pop of the same entry	Both apply; the net `refcnt` change is 0 and the entry stays live.
U2U allocate of a u_id being popped in the same cycle	Illegal; the u_id is still live at the start of the cycle (Section 8).

6.3 Timing diagrams

Figure 2 shows an allocation stalled by `full`. In cycle 0 the last free entry is consumed, so `full` is high in cycle 1 and `alloc_drdy` is low. The requester holds `alloc_srdy` and `alloc_u_id`. The pop in cycle 2 frees an entry, which is allocatable from cycle 3; `alloc_gen_id` then carries the freed ID (G1).

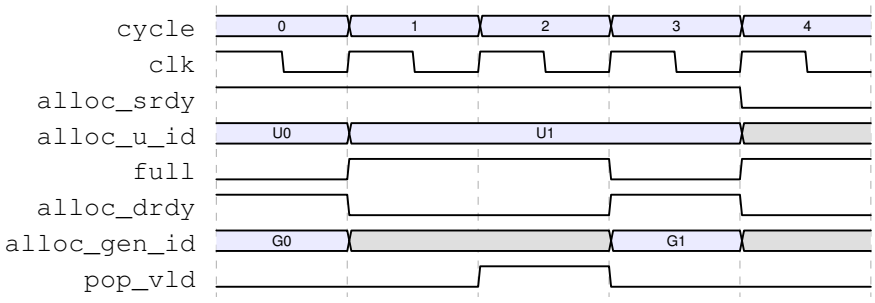


Figure 2. Allocation stalled by `full`, released by a pop.

Figure 3 shows a response by downstream ID. In cycle 0, `poll_d_port` returns the upstream ID (U2) for the response in the same cycle as the pop. In cycle 1 the entry is retired and the poll misses.

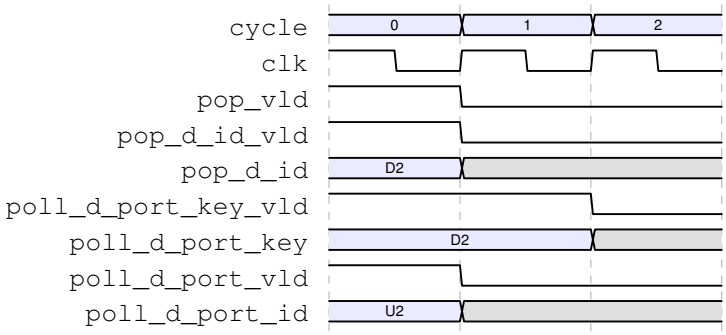


Figure 3. Response by downstream ID with a same-cycle poll.

7 Micro-architecture (informative)

This section describes a reference implementation. It is informative; any implementation that meets the requirements of Sections 3–9 is conforming.

7.1 Storage

```
entry[i] = { valid, u_id[U_ID_W-1:0], refcnt[REFCNT_W-1:0] }
             // refcnt is present in S2S mode only
d_id     == i
```

- `valid[NUM_ENTRIES-1:0]` is held as a flat register vector; the free pool is `~valid`.
- Entries with `i >= NUM_ENTRIES` (when `NUM_ENTRIES` is not a power of two) are tied off as valid (unavailable).

Free-entry counter and `full`. An internal counter `free_cnt` (`FREE_CNT_W` bits) tracks the number of free entries. It is not visible at the interface; its only external effect is `full`. Table 14 gives the update rules.

Table 14. Free-entry counter update rules.

Event in a cycle	`free_cnt` update
Allocate transfer that consumes an entry (U2U, S2U: every transfer; S2S: new u_id only)	-1
Pop that frees an entry (U2U, S2U: every pop; S2S: only when `refcnt` goes $1 \rightarrow 0$)	$+1$
S2S merge, or S2S pop that leaves `refcnt` > 0	no change
Both an entry-consuming allocate and an entry-freeing pop	no change

`full` is a registered flag, `full <= (free_cnt_next == 0)`, so a wide AND across `valid` is not required. `free_cnt` resets to `NUM_ENTRIES`. It must always equal the number of zero bits in `valid`.

7.2 Free-ID pool

Baseline. The pool *is* the inverted `valid` vector. A find-first-zero encoder produces the next free d_id on `alloc_gen_id`. This needs no separate storage and is inherently consistent with the table.

Alternative. For large `NUM_ENTRIES`, where the priority encoder is timing-critical, a free-ID FIFO pre-loaded with the values 0 to `NUM_ENTRIES-1` may be used: popped on allocate and pushed on retire. This changes the reuse order (FIFO instead of lowest-first) but not the function.

7.3 Lookup structures

- **d→u:** indexed read, `entry[poll_d_port_key].u_id`, gated by `valid`.
- **u→d:** parallel comparator (CAM) over all entries, `(entry[i].u_id == key) & valid[i]`, followed by an encoder for the hit index. Not instantiated in S2U (Section 3.3).
- The allocate path in U2U (duplicate check, assertion only) and in S2S (merge detect) reuses a CAM compare on `alloc_u_id`. Retire by u_id reuses a compare on `pop_u_id`. Up to three CAM instances (allocate, pop, `poll_u_port`) may be shared or duplicated depending on timing.
- **S2U needs no CAM at all.** Allocation, retire and lookup all use the d_id (entry index), so only indexed reads remain.

7.4 S2S reference counting

Each entry has a counter that is incremented on a merge and decremented on a pop. On allocation of a new u_id the counter loads 1. The entry is freed when a decrement takes the counter to 0. An increment and a decrement on the same entry in one cycle cancel.

8 Illegal Conditions

The behavior of the design under the conditions in Table 15 is **undefined**; an implementation is not required to recover, and users **shall not** rely on any particular behavior. It is recommended that each condition be checked by a simulation assertion. The design has no error outputs.

Table 15. Illegal conditions.

ID	Condition
I1	U2U: allocate of a u_id that is already live (including one being retired in the same cycle).
I2	Pop of an entry that is not live.
I3	`pop_vld` with both or neither of `pop_u_id_vld` and `pop_d_id_vld` set.
I4	S2U: `pop_u_id_vld` set.
I5	S2S: pop that would take `refcnt` below 0 (covered by I2).
I6	`alloc_srdy` dropped, or `alloc_u_id` changed, before `alloc_drdy` is seen.
I7	`pop_d_id` or `poll_d_port_key` $\geq$ `NUM_ENTRIES`.
I8	S2U: `poll_u_port_key_vld` set (u-keyed lookup is not supported).

Allocation when no entry is available is **not** illegal: `alloc_drdy` is low and the requester waits.

9 Reset

- While `rst_n` is low, all `valid` bits and all `refcnt` values **shall** be cleared and `free_cnt` **shall** load `NUM_ENTRIES`. The free pool then holds all `NUM_ENTRIES` d_ids and `full` is 0. The `u_id` storage need not be reset.
- While in reset, `alloc_drdy` **shall** be 0 and all poll results **shall** be 0.
- Inputs **shall** be ignored until `rst_n` has been high for one clock.

A Worked Examples (informative)

A.1 Conventions

- All examples use `NUM_ENTRIES = 4` (so `D_ID_W = 2` and the d_ids are d0 to d3) and `U_ID_W = 8`. Upstream IDs are written in hexadecimal without the `0x` prefix (`A0` is `0xA0`).
- Each trace row is one clock cycle. **Table** is the state at the *start* of the cycle. Allocations and pops in a cycle take effect in the next row.
- A dash (–) is a free entry. In S2S traces, `A0×2` is an entry holding u_id `A0` with `refcnt = 2`.
- **Poll** results are shown for the same cycle and are computed from the start-of-cycle table.
- **free** is the internal `free_cnt` at the start of the cycle. **full** is the output in that cycle.
- In the trace tables, `drdy` stands for `alloc_drdy`, `gen_id` for `alloc_gen_id`, `poll_u` for `poll_u_port` and `poll_d` for `poll_d_port`; `vld` and `id` are that port's result outputs. Signals not mentioned in a row are 0.

A.2 Side by side: the same requests in each mode

Two allocations of u_id A0, then one of B0, on an empty table (Table 16).

Table 16. The same requests in each mode.

	U2U	S2U	S2S
1st alloc A0	alloc_gen_id = 0	alloc_gen_id = 0	alloc_gen_id = 0, refcnt = 1
2nd alloc A0	**Illegal** (u_id already live)	alloc_gen_id = 1 (new entry)	alloc_gen_id = 0 (merge), refcnt = 2
alloc B0	alloc_gen_id = 1	alloc_gen_id = 2	alloc_gen_id = 1
Entries used	2 (A0, B0)	3 (A0, A0, B0)	2 (A0×2, B0×1)
Pops needed to free everything	2	3	3 (A0 twice, B0 once)
Allowed retire keys	u_id or d_id	d_id only	u_id or d_id
Table after the three allocations	[A0, B0, –, –]	[A0, A0, B0, –]	A0×2, B0×1, –, –

A.3 U2U trace

This trace covers allocate, allocate stalled by full, response by d_id, response by u_id, lookup hit and miss in both directions, and reuse of a freed ID (Table 17).

Table 17. U2U trace.

Cyc	Table (d0 d1 d2 d3)	free	full	Inputs	Outputs
0	– – – –	4	0	alloc A0	drdy=1, gen_id=0 (transfer)
1	A0 – – –	3	0	alloc B0; poll_u key A0	drdy=1, gen_id=1; poll_u: vld=1, id=0
2	A0 B0 – –	2	0	alloc C0; poll_d key 1	drdy=1, gen_id=2; poll_d: vld=1, id=B0
3	A0 B0 C0 –	1	0	alloc D0; poll_u key D0	drdy=1, gen_id=3; poll_u: vld=0 (pre-allocate state, miss)
4	A0 B0 C0 D0	0	1	alloc E0 (srdy held)	drdy=0, no transfer
5	A0 B0 C0 D0	0	1	alloc E0 (held); pop by d_id 1; poll_d key 1	drdy=0 (no pop→alloc bypass); poll_d: vld=1, id=B0
6	A0 – C0 D0	1	0	alloc E0 (held)	drdy=1, gen_id=1 (freed d1 reused)
7	A0 E0 C0 D0	0	1	pop by u_id A0; poll_u key A0	poll_u: vld=1, id=0
8	– E0 C0 D0	1	0	alloc F0; poll_u key A0; poll_d key 0	drdy=1, gen_id=0; both polls vld=0 (A0 retired)

Points illustrated:

- **Allocate (cycles 0–3).** The lowest free d_id is presented on alloc_gen_id in the same cycle as the handshake.

- **Same-cycle visibility (cycle 3).** The poll for D0 misses because polls see the start-of-cycle table; D0 is visible from cycle 4.
- **Stall (cycles 4–5).** With full = 1, alloc_drdy = 0 and the requester holds alloc_srdy and alloc_u_id. In cycle 5 the pop frees d1, but the allocation still waits one cycle.
- **Response by d_id (cycle 5).** pop_vld = 1, pop_d_id_vld = 1, pop_d_id = 1. The same cycle's poll_d on key 1 supplies the upstream ID B0 for the outgoing response.
- **Response by u_id (cycle 7).** pop_vld = 1, pop_u_id_vld = 1, pop_u_id = A0. The same cycle's poll_u on key A0 supplies d0.
- **Retired IDs miss (cycle 8).** Both polls miss, and d0 is immediately reusable.

A.4 S2U trace

This trace covers repeated allocation of one u_id receiving distinct d_ids, response by d_id only, and independent retirement (Table 18).

Table 18. S2U trace.

Cyc	Table (d0 d1 d2 d3)	free	full	Inputs	Outputs
0	- - - -	4	0	alloc A0	drdy=1, gen_id=0
1	A0 - - -	3	0	alloc A0 (same u_id)	drdy=1, gen_id=1 (new entry)
2	A0 A0 - -	2	0	alloc B0; poll_d key 1	drdy=1, gen_id=2; poll_d: vld=1, id=A0
3	A0 A0 B0 -	1	0	pop by d_id 0; poll_d key 0	poll_d: vld=1, id=A0
4	- A0 B0 -	2	0	alloc A0; poll_d key 0	drdy=1, gen_id=0 (lowest free); poll_d: vld=0
5	A0 A0 B0 -	1	0	pop by d_id 1	—
6	A0 - B0 -	2	0	poll_d key 0	poll_d: vld=1, id=A0 (the entry at d0 is still live)

Points illustrated:

- **Same u_id, distinct d_ids (cycles 0–1).** Two live entries hold A0. Each is retired individually by its d_id.
- **No u-keyed access.** Neither pop_u_id_vld nor poll_u_port_key_vld is used in S2U (Section 3.3, Table 15, I4 and I8). The response path always supplies the d_id received from downstream and reads the u_id from poll_d_port.
- **Reuse (cycle 4).** The d_id freed in cycle 3 is allocated again to a new A0.

A.5 S2S trace

This trace covers merge, reference counting, response by u_id and by d_id, and freeing when the count reaches zero (Table 19).

Table 19. S2S trace.

Cyc	Table (d0 d1 d2 d3)	free	full	Inputs	Outputs
0	- - - -	4	0	alloc A0	drdy=1, gen_id=0 (new entry, refcnt=1)
1	A0×1 - - -	3	0	alloc A0	drdy=1, gen_id=0 (merge, refcnt→2, no entry consumed)
2	A0×2 - - -	3	0	alloc B0; poll_u key A0	drdy=1, gen_id=1; poll_u: vld=1, id=0
3	A0×2 B0×1 - -	2	0	alloc A0	drdy=1, gen_id=0 (merge, refcnt→3)
4	A0×3 B0×1 - -	2	0	pop by u_id A0; poll_u key A0	poll_u: vld=1, id=0; refcnt→2, entry stays live
5	A0×2 B0×1 - -	2	0	pop by d_id 0; poll_d key 0	poll_d: vld=1, id=A0; refcnt→1
6	A0×1 B0×1 - -	2	0	pop by u_id A0; poll_d key 0	poll_d: vld=1, id=A0 (pre-pop); refcnt→0, entry freed
7	- B0×1 - -	3	0	alloc C0; poll_u key A0; poll_d key 0	drdy=1, gen_id=0 (freed d0 reused); both polls vld=0

Points illustrated:

- **Merge (cycles 1 and 3).** The same d_id is returned and free_cnt does not change.
- **Reference counting.** Three allocations of A0 need three responses. Popping by u_id and by d_id each reduce refcnt by one.
- **Free on zero (cycle 6).** free_cnt rises only on the pop that takes refcnt to 0.

S2S at full and at maximum count. Table 20 uses REFCNT_W = 2, so the maximum refcnt is 3. All four entries are live and full = 1.

Table 20. S2S at full and at maximum reference count.

Table at start	Inputs	alloc_drdy	Result
A0×1 B0×1 C0×1 D0×1	alloc A0	1	Merge accepted although full = 1: gen_id=0, A0 becomes A0×2; free_cnt stays 0.
A0×1 B0×1 C0×1 D0×1	alloc E0	0	New u_id and no free entry: stall.
A0×3 B0×1 C0×1 D0×1	alloc A0	0	refcnt at maximum: stall until a response decrements it.
A0×2 B0×1 C0×1 D0×1	alloc A0 **and** pop by u_id A0 in the same cycle	1	gen_id=0; the +1 and −1 cancel, so A0 stays A0×2 and the entry stays live.

A.6 Interface-level view of one transaction

A typical use of ip_idmap around a request/response path, in U2U mode:

1. **Request.** The requester drives alloc_srdy = 1 and alloc_u_id = 0x5A3. When alloc_drdy = 1, the request is forwarded downstream tagged with alloc_gen_id (for example 2).

2. **Response arrives** carrying downstream ID 2. In the same cycle:
   - `poll_d_port_key_vld = 1` and `poll_d_port_key = 2`: `poll_d_port_id = 0x5A3` is used as the upstream ID of the response.
   - `pop_vld = 1`, `pop_d_id_vld = 1` and `pop_d_id = 2` retire the entry.
3. **Next cycle.** d_id 2 is free and can be handed out again; a poll on `0x5A3` or on 2 now misses.